

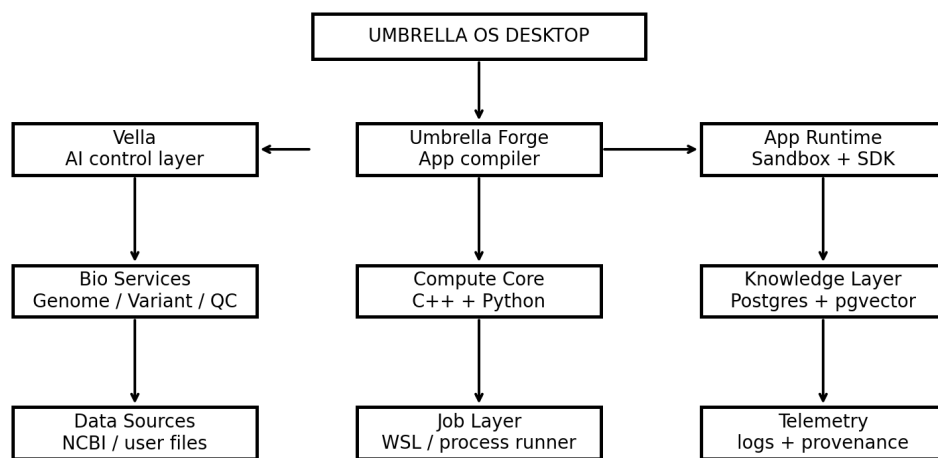
UMBRELLA

OS

AI-NATIVE COMPUTATIONAL BIOLOGY OPERATING SYSTEM

BUILD THESIS

Turn fragmented computational-biology workflows into one programmable scientific desktop: sequence analysis, mutation comparison, AMR surveillance, sample management, simulation, research, reporting and self-building scientific apps.



All scientific actions produce structured provenance and auditable results.

Umbrella OS high-level architecture: OS shell, Vella, Forge, app runtime, compute core, data and provenance.

Master build specification | Kiro Build Challenge | October 2026

1. Executive Summary

What is being built, why it exists, and what “done” means

Umbrella OS is a web-based scientific operating environment focused on computational biology. It keeps the recognizable desktop model - windows, app launcher, taskbar, terminal, files and a system control surface - but replaces generic productivity software with domain-native research applications. The central thesis is that scientific workflows should feel like one coherent computer rather than a collection of disconnected command-line tools.

The primary differentiator is Umbrella Forge. A scientist can request a small tool that does not exist, such as “build a sequence comparator with a GC-content panel,” and the system converts the request into a constrained application specification, generates UI and logic against the Umbrella SDK, checks permissions and validation rules, then installs the result into the desktop. The generated application is treated as a first-class app with a manifest, version, permissions, state schema and provenance.

Vella is the scientific control layer. It interprets natural-language requests, selects tools, invokes analysis jobs, retrieves evidence, coordinates apps and explains results. It should never be the scientific source of truth: deterministic engines, versioned databases and model outputs remain the authoritative layers; Vella narrates and orchestrates them.

Layer	Purpose	MVP implementation
Desktop shell	Window manager, launcher, taskbar, app registry	Next.js + React + TypeScript
Vella	Natural-language orchestration and explanation	LLM gateway + tool calling + RAG
Umbrella Forge	Generate/evolve safe scientific apps	App spec DSL + templates + validator
Bio Core	FASTA parsing, QC, mutation, k-mer/AMR features	Python + C++17 hot paths
Data plane	Metadata, raw sequences, derived tables	PostgreSQL + Parquet/DuckDB + filesystem/S3
ML layer	Phenotype prediction and confidence	scikit-learn; optional PyTorch embeddings
Jobs	Long-running analyses	FastAPI worker subprocesses / WSL
Provenance	Versions, parameters, evidence and hashes	Postgres tables + JSON manifests

MVP DEFINITION

A judge can upload a real bacterial genome, run QC, inspect mutations, execute AMR analysis, view a model-backed phenotype prediction, ask Vella to explain the result, and create a small new scientific app through Forge.

2. The Real Problem

Why a scientific operating system is useful

Computational biology is powerful but fragmented. A researcher may move between FASTA/FASTQ files, web tools, AMR annotators, custom scripts, databases, notebooks, command-line programs and spreadsheets. The problem is not a lack of individual tools. The problem is the gap between those tools: data must be converted, parameters remembered, results copied, versions tracked and conclusions reconstructed manually.

Umbrella OS treats this as an integration and reproducibility problem. A sample should have one persistent identity. Every analysis should point back to the same input bytes, the same tool/version, the same parameters and the same output artifact. The interface should make that chain visible instead of forcing the scientist to remember it.

Primary users

User	Need	Umbrella response
Student / builder	Learn genomics by running visible workflows	Interactive apps + terminal + Vella
Computational biologist	Batch analysis and reproducible results	Jobs, provenance, analysis studio
Researcher	Compare samples, variants and evidence	Sample Vault + Variant Explorer
AMR analyst	Screen genes/mutations and connect to phenotype	AMR Sentinel + prediction model
Lab / project lead	Understand dataset coverage and QC	Dashboard + provenance + reports

Design constraints

- Phase 1 is bacterial-genome focused because the organizer-pinned benchmark and BV-BRC phenotype linkage are bacterial. Virus and fungal support is an extensible architecture, not a promise that the MVP trains on those domains.
- The system is a computational research tool, not a clinical diagnostic device and not an autonomous wet-lab protocol generator.
- Every model output must expose evidence, dataset version, model version and out-of-distribution warnings where possible.
- The 18.5-hour sprint is a vertical slice. Heavy data preparation and large-scale training happen before/after the event only where the challenge rules permit; the submitted implementation should be built during the allowed window.

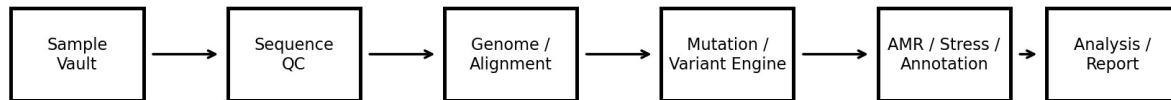
PROBLEM STATEMENT

Researchers need a single environment that can turn raw bacterial sequence data into reproducible analysis, comparative evidence and model-backed insights without stitching together disconnected tools by hand.

3. Product Thesis and User Journey

From sample to scientific result

UMBRELLA SCIENTIFIC WORKFLOW



Every result retains source, tool/version, parameters and provenance.

Umbrella scientific workflow: Sample Vault -> QC -> genome/alignment -> mutation/variant -> AMR/stress/annotation -> analysis/report.

The core user journey is deliberately linear even though the underlying platform is modular. The user starts with a sample. Umbrella assigns or imports an identity. QC establishes whether the input is usable. Genome analysis establishes basic sequence facts. Mutation and variant tools compare sequences or inspect selected loci. AMR analysis adds resistance evidence. The analysis/reporting layer turns the resulting objects into a research artifact with provenance.

The system should feel “stateful”

When the user opens Sample 01827, they should not re-upload its FASTA file in every app. The Sample Vault points to the immutable input artifact. Apps query that identity and produce linked outputs. A new report is a view over the same state, not a new copy of the entire analysis.

Object	Persistent fields	Examples
Sample	id, source, organism, accession, input hash, created_at	sample_01827
Sequence	sample_id, contig, length, sequence_uri, checksum	contig_4
Analysis	id, tool, version, params, status, output_uri	qc_0041
Variant	position, ref, alt, type, evidence, source	SNP at coordinate x
AMR hit	gene/mutation, drug class, identity, coverage, tool_version	bla...
Prediction	antibiotic, probability, model_version, feature summary	ciprofloxacin
Report	title, inputs, analyses, generated_at, checksum	report_2026_10_03

DESIGN RULE

Every scientific object should be linkable from the UI and serializable to JSON. If an app cannot explain where its data came from, it is not a first-class Umbrella app.

4. Desktop and UI System

The operating system should feel like an operating system

Build the interface as a real desktop shell rather than a dashboard with a sidebar. The desktop has a taskbar, app launcher, open-window state, window focus, draggable/resizable panels where practical, a system tray/control center, notifications, file browsing and a persistent Vella command surface. Scientific apps open as windows so the judge can switch between them while keeping analyses active.

Desktop regions

Region	Behavior	Implementation
Taskbar	Open apps, active window, system controls	React state + Framer Motion
App Drawer	Search installed apps and generated apps	App Registry API
Desktop	Pinned applications / recent samples	Local preference store
Window Manager	Open, close, minimize, focus, resize	Client-side window state
Control Center	Compute status, active jobs, permissions	WebSocket/SSE status feed
Command Bar	Global Vella interaction	Streaming assistant panel

Visual language

- Graphite/near-black base, dark green as system identity, restrained gold for active states, red only for warnings/errors.
- Use typography that distinguishes narrative text from scientific identifiers and terminal output.
- Charts should prioritize legibility over decoration: axes, units, dataset version and sample identity are always visible.
- A scientific result must never be hidden behind decorative animation. Motion is for state transitions and system feedback.

Core reusable components

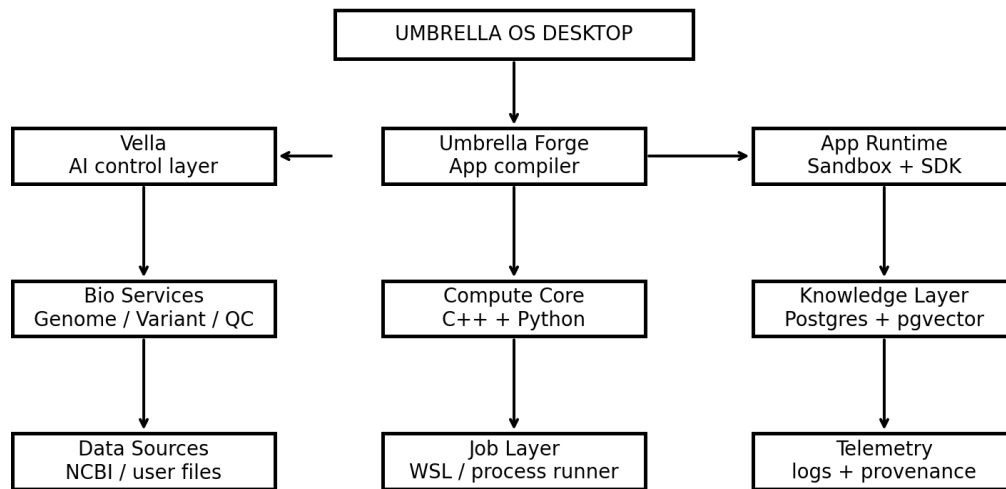
Create a small design system first: Window, Panel, DataTable, Metric, Badge, Timeline, EvidenceLink, SequenceTrack, Heatmap, ConfidenceBar, JobProgress, TerminalPane, PermissionDialog and AppManifestCard. Generated apps should be assembled from these components through the App SDK instead of inventing arbitrary UI.

JUDGE MOMENT

Open five native scientific apps, drag two windows side by side, select the same sample in each, then switch back to Vella and ask one question about all of them. The “OS” claim becomes visually obvious.

5. System Architecture

How the pieces fit together



All scientific actions produce structured provenance and auditable results.

Reference architecture for the Umbrella OS platform.

The architecture uses a thin web shell over a modular service layer. Next.js owns the desktop experience and server-side routing. FastAPI exposes typed scientific job APIs. The Bio Core contains deterministic sequence-processing functions and CLI engines. The data plane stores raw bytes separately from analytical metadata. Vella calls the same APIs that the UI uses; it does not bypass the service boundary.

Request path

```

User -> Next.js UI -> FastAPI
|
+--> App Registry / Permissions
+--> Scientific Job API
+--> Data Catalog
+--> Model Service
+--> Vella tool adapter
  
```

Long job -> WSL / subprocess -> artifact store -> job event stream -> UI

Repository layout

```

umbrella-os/
apps/web/      # Next.js desktop
services/api/  # FastAPI
services/ai/   # Vella + tool routing
core/bio/      # Python scientific primitives
core/cpp/      # optional hot-path C++17
tools/amr/     # AMRFinderPlus adapter
ml/           # features, train, evaluate, serve
data/schemas/ # JSON/Parquet schemas
packages/app-sdk/ # generated app contracts
packages/ui/   # shared desktop + scientific UI
.kiro/specs/   # feature specs
.kiro/steering/ # project rules
.kiro/hooks/   # validation automation
  
```

ARCHITECTURE RULE

Do not let generated apps call the host filesystem or shell directly. They call declared capabilities through the App Runtime.

6. Technology Stack

Recommended stack for the challenge and the longer-term platform

Area	Technology	Why	Sprint priority
Desktop	Next.js 15 + React 19 + TypeScript	Fast UI iteration, routing, component ecosystem	P0
Styling	Tailwind CSS + shadcn/ui + Framer Motion	Consistent system UI and quick polished interactions	P0
Plots	Apache ECharts	Genome/feature charts without heavy custom SVG work	P0
Terminal	xterm.js	Real terminal pane and command output	P1
API	FastAPI + Pydantic + Uvicorn	Typed scientific APIs and Python integration	P0
Data analytics	DuckDB + Parquet / PyArrow	Fast local columnar analytics, easy batch work	P0
Metadata	PostgreSQL	Transactions, sample registry, jobs, provenance	P0
Raw data	Filesystem during sprint; S3/MinIO later	Large FASTA/FASTQ artifacts should not live in Postgres	P0
ML	scikit-learn + PyTorch	Baseline plus optional neural models	P0/P2
Bio tools	AMRFinderPlus; optional ResFinder/cAMRah/XTree	Use established scientific tools where appropriate	P0/P2
Core acceleration	C++17 / Rust only for hotspots	Reuse systems skill without making the whole stack complex	P2
Execution	WSL2 Ubuntu + Docker	Reproducible Linux toolchain on Windows	P0
Dev AI	Kiro Specs, Steering, Hooks, MCP	Challenge-native engineering workflow	P0

Why not 10 languages?

You can write C, C++, C#, Java, Ruby, Rust, Python, JavaScript, TypeScript, Fortran and COBOL, but the architecture should not use ten languages just to prove it can. Use TypeScript for the OS, Python for scientific orchestration/ML, and C++17 only where a measured hot path benefits. Add Rust later if a component needs stronger memory safety or concurrency isolation. Language variety is a capability of the terminal, not a reason to fragment the production codebase.

PERFORMANCE PRINCIPLE

Measure first. Start with Python for correctness and iteration. Promote only the bottleneck - for example k-mer scanning or sequence diffing - into C++17 behind a stable CLI/JSON contract.

7. Vella - Scientific Control Layer

Assistant, orchestrator, explainer

Vella should have three operating modes: Explain, Analyze and Orchestrate. Explain answers questions about an existing result using the evidence store. Analyze calls deterministic analysis tools and models, then narrates the output. Orchestrate sequences multiple tools and apps into a workflow. This mirrors the original Synapse philosophy of separating generative, inference and agentic behavior.

Mode	Example request	Backend path
Explain	"Why is this sample flagged?"	Retrieve result + evidence -> grounded response
Analyze	"Check mutations in sample 1827."	Mutation service -> stored result -> summary
Orchestrate	"Run QC, AMR and prediction on this sample."	Create job graph -> execute -> collect artifacts

Tool contract

```

Tool: genome_qc
Input: {sample_id, options}
Output: {job_id}

Tool: mutation_compare
Input: {reference_sample, query_sample}
Output: {job_id}

Tool: amr_scan
Input: {sample_id, organism, database_version}
Output: {job_id}

Tool: phenotype_predict
Input: {sample_id, antibiotic_set, model_version}
Output: {job_id}

```

Vella must be grounded by returning tool outputs with explicit provenance. The LLM should not invent mutation coordinates, AMR genes, model probabilities, database versions or scientific citations. If a tool fails, Vella should report the failure instead of completing a plausible-sounding answer.

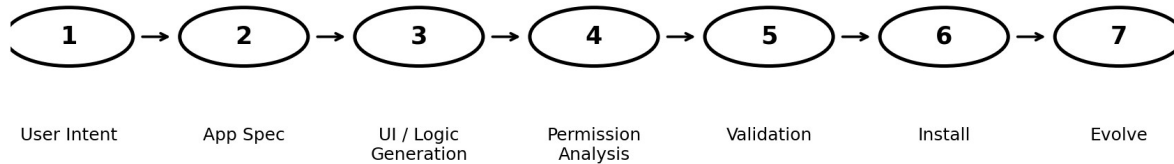
Voice

A voice-first interface can be included, but voice should be a convenience layer rather than the sole security boundary. For a real system, use a normal authenticated session/passkey plus optional speaker verification signal. The hackathon implementation should show the concept without claiming biometric-grade authentication performance.

8. Umbrella Forge

Self-building scientific applications

UMBRELLA FORGE - SELF BUILDING APPLICATION PIPELINE



Apps are generated against a controlled SDK/runtime, never executed as arbitrary model output.

Forge pipeline: intent -> app spec -> UI/logic -> permissions -> validation -> install -> evolve.

Forge is the feature that turns Umbrella from an application suite into an extensible operating environment. The user does not ask the AI for source code. The user requests a capability. Forge converts that capability into a constrained Application Specification and builds against the Umbrella SDK.

Forge pipeline

1. Intent extraction: identify purpose, screens, data, actions, dependencies and required capabilities.
2. Spec generation: write a machine-readable app manifest and human-readable spec.
3. UI/logic generation: map the spec onto approved components and state/actions.
4. Permission analysis: request only the capabilities required by the app.
5. Validation: schema, type, permission, runtime and test checks.
6. Install: register version, icon, manifest and artifact.
7. Evolve: later prompts create a modification spec and versioned update rather than destroying the previous app.

SAFETY ARCHITECTURE

Generated apps are declarative by default. Advanced apps may run sandboxed React/TypeScript, but only through a restricted SDK and explicit capability tokens. Never evaluate raw model output as arbitrary host code.

9. App Runtime and SDK

How a generated app becomes a real OS application

The runtime is the real technical innovation behind Forge. It defines a small application language: layout components, data bindings, actions, events, state and capabilities. This lets a model “program” the OS without getting unrestricted control of the machine.

Manifest

```
{
  "id": "sequence-comparator",
  "version": "1.0.0",
  "entry": "native",
  "permissions": ["sample.read", "analysis.run"],
  "data": ["sample", "sequence", "variant"],
  "components": ["sequence_view", "diff_table", "metric_card"],
  "actions": ["compare", "export"],
  "provenance": {"generator": "vella", "spec": "..."}
}
```

Core SDK components

Component	Purpose
Window	OS-level container and lifecycle
DataTable	Sortable scientific records
SequenceTrack	Colored bases/regions/coordinates
Heatmap	Variant/feature density
Chart	QC, phenotype and model metrics
JobProgress	Long-running tool execution
EvidenceLink	Open source artifact/tool/version
PermissionGate	Capability-aware UI action
FilePicker	Controlled sample/artifact selection
ReportBlock	Composable report output

Inter-app communication

Apps communicate through typed events and data contracts rather than direct component references. Example: Genome Analyzer publishes `sample.selected`; Mutation Lab consumes it. Mutation Lab publishes `variants.created`; Analysis Studio subscribes. Vella can inspect the registry and call app capabilities through the same event bus.

10. Data Strategy and Benchmarking

Use the organizer dataset first

The organizer-provided benchmark should be the canonical event dataset. The supplied brief indicates a fixed challenge dataset derived from real bacterial genome data and laboratory resistance results, designed to let teams focus on modeling rather than collecting large raw archives. The report below also gives a reproducible external-data path using BV-BRC and related tools.

BV-BRC: what the API exposes

BV-BRC exposes `genome` records and a `genome\_amr` data type. The AMR record includes genome identifiers, antibiotic name, evidence type, laboratory typing method/platform, raw measurement, measurement sign, measurement unit/value, resistant phenotype, testing standard/year, source and provenance fields. This distinction is important because the challenge brief warns not to train on general phenotype fields that may contain predictions. Use the laboratory-measured records pinned by the organizers. [1]

Field family	Store	Use
Identity	genome_id, accession, taxon_id	Join genome to sequence and metadata
Lab measurement	measurement, sign, unit/value	Regression target or audit trail
Phenotype	resistant_phenotype	Classification target when organizer-pinned
Standard	testing_standard, year	Do not mix incompatible breakpoint interpretations
Method	laboratory_typing_method/platform	Stratify or audit method effects
Evidence	evidence, source, computational_method	Filter measured vs predicted evidence

Recommended data hierarchy

- Level A: organizer-pinned train/validation/test dataset.
- Level B: BV-BRC public genomes and laboratory AMR records for extension and stress testing.
- Level C: AMRFinderPlus/ResFinder/cAMRah outputs as annotation features and tool-comparison evidence, not as ground-truth phenotype labels.
- Level D: optional Kaggle mirrors only when source, license and version are traceable back to the original resource.

DATA RULE

Never use a predicted phenotype from a model/tool as the target for training a second model unless the task explicitly calls for teacher-student or weak supervision. The default target is organizer-pinned laboratory measurement/phenotype.

11. Data Acquisition

Exact practical workflow

For the event, prefer the provided dataset so you do not spend the sprint downloading hundreds of gigabytes. For reproducible development, build an acquisition adapter that can pull BV-BRC metadata via HTTP and sequence files through the supported APIs or a bulk-download workflow. BV-BRC documents GET/POST queries and multiple return formats, including JSON, CSV/TSV and FASTA for genome sequences. [1][2]

Metadata query example

```
curl -H "Accept: text/tsv" \
  "https://www.bv-brc.org/api/genome_amr?eq(evidence,Phenotype)&eq(public,true)&limit(1000)" \
  > genome_amr.tsv
```

Sequence retrieval example

```
curl -H "Accept: application/dna+fasta" \
  "https://www.bv-brc.org/api/genome_sequence/?eq(genome_id,GENOME_ID)&limit(1000)" \
  > sample.fasta
```

The exact query should be narrowed by the organizer-pinned genome IDs, antibiotics and evaluation split. Do not “discover” the test set by querying the same public filters used for training. The benchmark manifest should be treated as immutable.

Local acquisition manifest

```
dataset_manifest.json
{
  "source": "BV-BRC",
  "retrieved_at": "2026-10-03T...Z",
  "query": "...",
  "records": 15000,
  "sha256": "...",
  "sequence_bytes": "...",
  "notes": "organizer benchmark / local reproduction"
}
```

For a very large corpus, keep raw FASTA immutable and create derived Parquet tables for metadata, AMR annotations, k-mer statistics and model features. Do not copy full sequences into relational rows.

12. Large Dataset Layout

How to handle 120 GB without turning Postgres into a disk

Treat large genomic data as an object store plus columnar metadata. A 120 GB corpus is manageable on an SSD/NVMe-backed WSL environment if the raw sequences are immutable and the derived tables are compact. During the challenge, only the organizer subset should be staged locally unless the data is already available.

```
data/
raw/
  bvbrc/
    genome/      # FASTA/GZIP
    amr/         # TSV/JSON
bronze/         # minimally normalized records
silver/
  samples.parquet
  amr_features.parquet
  variants.parquet
  qc.parquet
gold/
  train.parquet
  validation.parquet
  test_manifest.parquet
models/
  lr/
  embedding/
artifacts/
reports/
jobs/
```

Storage	Use	Rule
Raw object store	FASTA/FASTQ, original AMR TSV	Immutable, hashed, versioned
PostgreSQL	Sample registry, jobs, provenance, app state	Metadata only
Parquet	Feature matrices and batch tables	One row per sample/antibiotic or sample/feature
DuckDB	Local analysis and joins	Read Parquet directly
Model store	Joblib/ONNX/PyTorch weights	Version with feature schema hash
Cache	Intermediate tool outputs	Evictable, never source of truth

Compression

Use gzip/zstd for transport and archive storage. For analysis tables, Parquet with dictionary encoding and compression is preferred. For sequence chunks, store raw FASTA once and generate indexed or chunked representations only when a downstream algorithm benefits from them. The goal is to avoid a pipeline that expands the corpus repeatedly into many redundant copies.

13. Sequence QC and Normalization

Make dirty inputs explicit

Before any model training, create a deterministic QC report per sample. The report should not silently “fix” a genome. It should record what was normalized and what was discarded. This protects the benchmark from hidden preprocessing differences.

QC metric	Computation	Action
Sequence length	sum non-header bases	Flag implausible or empty sequences
Ambiguous bases	count N/non-ACGT symbols	Record percentage; optionally mask for feature extraction
GC content	(G+C)/valid bases	Store numeric metric
Contig count	FASTA record count	Store numeric metric
Duplicate contigs	hash / sequence duplicate check	Flag
Malformed FASTA	parser validation	Fail fast
Input hash	SHA-256 raw file	Use as immutable identity

```
def normalize_sequence(raw):
    # keep transformation deterministic and recorded
    seq = raw.upper()
    seq = seq.replace(" ", "").replace("\n", "")
    allowed = set("ACGTN")
    seq = "".join(base if base in allowed else "N" for base in seq)
    return seq
```

QC output

```
{
  "sample_id": "...",
  "sha256": "...",
  "total_bases": 5210341,
  "gc_fraction": 0.508,
  "n_fraction": 0.00003,
  "contigs": 18,
  "status": "PASS",
  "transform": "uppercase + whitespace removal + non-ACGTN -> N"
}
```

QC is both a scientific control and a software feature. The UI should surface a red warning when a model is being asked to score a sample that is outside the training QC distribution.

14. Annotation Pipeline

AMRFinderPlus first; harmonize evidence, do not confuse it with ground truth

NCBI AMRFinderPlus identifies AMR genes and resistance-associated point mutations from protein annotations and/or assembled nucleotide sequence, using a curated Reference Gene Database and curated HMM collection. The software and databases are openly available, and the current public release shown by the project repository is v4.2.7. [3][4]

Canonical event command

```
amrfinder --plus \
--nucleotide sample.fna \
--output sample.amrfinder.tsv \
--threads 8
```

When annotated protein, GFF and nucleotide files are available, combined mode can be richer because AMRFinderPlus can connect matches to gene coordinates. It also has options for organism-specific mutation screening. Pin the exact software version and database version in your provenance record; the database is released separately from the executable. [4]

Optional tool comparison

ResFinder provides another AMR detection route, while cAMRah is a reproducible WDL-based workflow that combines multiple AMR-finding tools and harmonizes their outputs. cAMRah currently documents combinations involving AMRFinderPlus, ResFinder, RGI/CARD, Abricate/NCBI, Abricate/ARG-ANNOT and BV-BRC. [5]

Layer	What to store
Raw tool output	Original TSV/JSON, version, command, database version
Normalized hit	sample, locus, gene, class, identity, coverage, mutation
Evidence link	tool output row + source database
Conflict	tool A vs tool B disagreement, not silently resolved
Feature	binary/count/quality feature used by model

SCIENTIFIC BOUNDARY

AMRFinderPlus/ResFinder annotations are features/evidence. The phenotype target must come from laboratory measurements or the organizer-pinned phenotype label, not from the annotator itself.

15. DNA Feature Engineering

Turn genomes into trainable representations

Use a layered feature strategy so the baseline is explainable and the neural extension is optional. The first feature family comes from curated AMR annotations; the second comes from sequence-derived k-mers; the third comes from genomic summary statistics; the fourth is optional learned embeddings.

Feature family	Representation	Benefit	Cost
AMR genes	binary presence/absence; optional allele IDs	Interpretable and fast	Low
AMR mutations	binary mutation indicators	Captures known resistance loci	Low
k-mers	hashed 21/31-mer presence or counts	Sequence-only signal beyond annotations	Medium
Genome stats	length, GC, contigs, N fraction	Controls / covariates	Low
Taxonomy	one-hot / target-encoded lineage	Captures domain shift; avoid leakage	Low
Embeddings	DNABERT-2/HyenaDNA chunk embeddings	Learned context signal	High

K-mer matrix

For a CPU-friendly baseline, transform each genome into a sparse feature vector. A simple approach is canonicalizing each k-mer against its reverse complement, hashing to a fixed dimension and storing counts or binary presence. Use a dimension such as 2^{18} to 2^{22} depending on memory and collision tolerance; tune on the validation set.

```

for sequence in genome:
    for kmer in rolling_kmers(sequence, k=31):
        if "N" not in kmer:
            key = canonical(kmer, reverse_complement(kmer))
            idx = hash64(key) % D
            X[idx] += 1
  
```

Do not interpret a single k-mer coefficient as a biological causal mechanism. It is a statistical feature. For the explainable AMR model, prioritize curated genes/mutations as named evidence and show k-mer features only as “sequence-derived signals.”

16. Targets and Label Construction

What exactly the model learns

The dataset contains laboratory measurements such as MIC and may also contain standardized phenotype labels. BV-BRC documents `measurement`, `measurement\_sign`, `measurement\_unit`, `measurement\_value`, `resistant\_phenotype`, `testing\_standard` and `testing\_standard\_year` as separate fields. [1] That separation is crucial.

Two model families

Task	Target	Recommended model
Phenotype classification	organizer-pinned Resistant / Intermediate / Susceptible	multiclass logistic regression
Binary decision	Resistant vs non-resistant only when benchmark defines it	binary logistic regression
MIC regression	log2-transformed MIC or equivalent normalized scale	regularized regression / gradient boosting
Uncertainty	calibrated probability + OOD flag	Platt/isotonic + distance metric

MIC handling

Normalize units per antibiotic before modeling. Preserve censoring symbols such as `<` and `>`; they do not mean exact numeric measurements. For a sprint baseline, if the challenge target already supplies categorical phenotype labels, train on those labels and treat raw MIC as an auxiliary analysis field. For a regression experiment, use an interval-aware objective where practical, or perform a clearly documented boundary-imputation sensitivity analysis.

Avoiding phenotype leakage

- Do not include `resistant\_phenotype` or a field derived from it among features.
- Do not include a post-hoc model prediction from another tool as a feature unless explicitly framed as stacked modeling and fit only on training folds.
- Do not use testing-standard-derived labels from the held-out test set to tune thresholds.
- Keep the organizer test manifest sealed and versioned.

17. Baseline ML Model

Per-antibiotic regularized logistic regression

The recommended core model is one regularized logistic-regression model per antibiotic. This is fast, calibratable and explainable, and matches the organizer guidance. The model can ingest the AMRFinderPlus features plus selected genomic summary statistics and optionally sparse k-mer features.

Training recipe

1. Build the feature matrix only from the training partition.
2. Impute/standardize numeric features using training-fold statistics.
3. Use class weights when the phenotype classes are imbalanced.
4. Fit logistic regression with L2 regularization; tune C on validation data.
5. Calibrate probabilities on a validation split only.
6. Freeze the pipeline, then evaluate once on the sealed test set.
7. Serialize model + preprocessing + feature schema + dataset manifest hash.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

model = Pipeline([
    ("scale", StandardScaler(with_mean=False)),
    ("clf", LogisticRegression(
        max_iter=2000, class_weight="balanced",
        penalty="l2", C=1.0, solver="liblinear"))
])
model.fit(X_train, y_train)
```

Explainability view

For each prediction, display the strongest positive and negative feature contributions. For curated AMR genes/mutations, map coefficients to human-readable names. For sparse sequence features, aggregate at the feature-family level: “sequence-derived k-mer signal” rather than a long list of meaningless hashes.

DEMO OUTPUT

For each antibiotic show: predicted class, calibrated probability, evidence genes/mutations, data quality, model version, and an explicit “research use only” note.

18. Neural DNA Model

Optional second layer: learned sequence representations

A neural model is useful as a stretch goal, but do not make it the only working path. HyenaDNA demonstrated long-context genomic modeling up to 1 million single-nucleotide tokens, while DNABERT-2 introduced an efficient BPE tokenization strategy for multi-species genome understanding. These are pretrained research models; the sprint should reuse pretrained weights rather than attempt foundation-model pretraining from scratch. [6][7]

Practical architecture

```
Genome FASTA
-> chunk into 4k-16k bp windows
-> encoder (DNABERT-2 or HyenaDNA)
-> chunk embedding
-> attention / mean pooling over chunks
-> concatenate curated AMR features
-> per-antibiotic classifier
-> calibration
```

For an 18.5-hour prototype, freeze the pretrained encoder and train only a small pooling/classification head. This keeps compute manageable and makes the experiment reproducible. If GPU time is scarce, use offline embeddings for a subset and compare them to the logistic baseline.

Small model alternative

If you want a model you actually train from scratch during the sprint, build a compact 1D CNN over fixed-length sequence windows: one-hot encode A/C/G/T/N, apply several 1D convolution blocks, global average pooling, concatenate QC features, and train one head per antibiotic. It will be weaker than a full genomic foundation model but it is genuinely trained end-to-end on your data and easier to debug.

Option	Training from scratch?	Sprint fit
Logistic on AMR features	Yes	Excellent
Small 1D CNN	Yes	Good
DNABERT-2 frozen embeddings	No encoder; train head	Good stretch
HyenaDNA frozen embeddings	No encoder; train head	Good but heavier
New genome foundation model	No	Not a sprint task

19. Evaluation and Leakage Control

Make the model believable

Genomic datasets can contain near-duplicate or closely related isolates. A random row split can therefore overstate performance because the model may see nearly the same genome lineage in training and test. The safest approach is grouped splitting by biological identity or similarity cluster, using the organizer's grouping guidance where supplied.

Split hierarchy

1. Group identical assembly/accession/sample duplicates together.
2. If possible, cluster close genomes using a reproducible sketch/ANI approach and keep clusters intact across splits.
3. Perform train/validation/test partitioning on groups, not individual rows.
4. Stratify by antibiotic and phenotype where possible so rare classes are represented.
5. Freeze the test set after the first model is chosen.

Metric	Use	Why
ROC-AUC	ranking quality	Threshold-independent
PR-AUC	imbalanced resistance task	More informative when positives are rare
Balanced accuracy	multiclass / imbalance	Equal class importance
Macro F1	class-wise quality	Prevents majority-class dominance
Brier score	probability calibration	Checks whether confidence matches outcomes
MAE/RMSE on log2(MIC)	MIC regression	Magnitude of error

External validity

Report performance by species/genus and by testing standard where enough samples exist. A model that performs well on E. coli but fails on a different lineage should not be presented as universally generalizable. The UI should show the evaluation population and training coverage alongside every model version.

TRUST PRINCIPLE

Do not turn one scalar "trust score" into a fake scientific certainty. Show separate research-confidence, evidence coverage and OOD indicators.

20. Confidence, Evidence and OOD

What the user should see beside every prediction

Umbrella should distinguish three different ideas: model probability, evidence coverage and distributional similarity. A calibrated probability is not the same thing as evidence quality, and neither is the same as whether the sample resembles the training population.

Signal	Meaning	UI label
Calibrated P(class)	Model probability after calibration	Model confidence
Evidence coverage	How much expected evidence is actually available	Evidence coverage
OOD distance	Similarity to training feature distribution	In-domain / review
QC state	Sequence quality relative to training set	Data quality
Provenance completeness	Version/hash/parameter completeness	Traceability

Research Trust Index (optional UI composite)

For a visual score, use a clearly labeled composite UI index that never claims clinical validation. Example: $RTI = 0.45 \cdot \text{calibrated confidence} + 0.20 \cdot \text{evidence coverage} + 0.20 \cdot \text{in-domain score} + 0.15 \cdot \text{QC score}$. Display the components beneath it so the user can see what actually drives the number. The formula is a product-design convention, not a scientific standard.

OOD implementation options

- Feature-space Mahalanobis distance for dense embeddings.
- Nearest-centroid or k-nearest-neighbour distance for sparse curated features.
- Isolation Forest on the training feature distribution for a cheap warning signal.
- Species/lineage coverage check: if the sample belongs to a poorly represented group, downgrade the visual confidence flag.

The final prediction card should therefore look more like an evidence dossier than a single score.

```

PREDICTION
Antibiotic: example
Class: Resistant
Model confidence: 0.91 (calibrated)
Evidence coverage: 0.84
OOD flag: LOW
QC: PASS
Model: amr-lr-v0.1.0
Data: organizer-benchmark-2026.10
  
```

21. Native App #1 - Genome Analyzer

The entry point for sequence analysis

Genome Analyzer accepts FASTA/FASTQ-derived assemblies and shows the basic state of a sample before deeper analysis. It should be fast enough to respond immediately on small files and switch to a background job for large inputs.

Screen	What it does
Overview	length, contigs, GC%, N%, checksum, organism/accession
Sequence browser	contig list, coordinate view, selected region
QC panel	warnings, distributions and pass/fail state
Annotations	links to AMR/feature outputs when available
Run analysis	launch QC, AMR, mutation or report jobs

Implementation

The FastAPI endpoint creates a `sample` record, stores the raw file in the artifact store, computes a SHA-256 checksum, and creates a QC job. The frontend subscribes to job progress via Server-Sent Events or polling. Results are persisted in a `qc\_result` table keyed by `sample\_id` and `analysis\_id`.

```
POST /api/samples
POST /api/samples/{id}/qc
GET /api/samples/{id}
GET /api/analyses/{id}
GET /api/jobs/{id}/events
```

Implementation tip

Do not send megabytes of sequence content through the browser just to render a dashboard. Store the file server-side and fetch slices by coordinate when the user zooms into a region.

DEMO

Upload a bacterial FASTA, watch the QC job complete, open the sequence browser and click “Analyze with AMR Sentinel.” The next app should open with the same sample already selected.

22. Native App #2 - Mutation Lab

Reference vs sample, with auditable differences

Mutation Lab compares a query sequence or genome to a selected reference. It should support a fast exact/near-exact path for short sequences and an adapter to a mature whole-genome alignment tool for large assemblies. The UI should distinguish substitutions, insertions, deletions and larger structural differences where the selected tool can support them.

Panel	Function
Reference selector	Choose sample/reference genome
Query selector	Choose comparison sample
Difference summary	Counts by variant type
Coordinate table	position, ref, alt, contig, source
Sequence viewer	show bases around selected variant
Export	VCF-like table/JSON + provenance

MVP implementation

For the sprint, implement a deterministic pairwise comparison for a selected contig or short region. Use an established edit-distance/alignment library or a simple dynamic-programming implementation where input size is bounded. Whole-genome acceleration can be delegated to MUMmer/minimap2-like tools in a post-sprint extension.

Link to AMR

If a mutation coordinate overlaps an AMRfinderPlus resistance-associated mutation or another curated annotation, Mutation Lab should show an evidence link rather than making an independent claim. The user can jump from the mutation row to the AMR Sentinel record.

SAFETY BOUNDARY

Mutation Lab is an analysis/comparison tool. It should not generate operational wet-lab instructions for modifying pathogenic organisms.

23. Native App #3 - AMR Sentinel

The flagship successor to Genome Firewall

AMR Sentinel turns the existing Genome Firewall concept into a native OS application. The original engineering advantage - fast sequence ingestion, indexed AMR matching, mutation-aware detection and concurrent scoring - becomes a reusable compute service inside Umbrella Core.

Screen	What the user sees
Run overview	sample, organism, tool/database versions, runtime
Resistome	detected genes and mutations
Drug panel	per-antibiotic evidence summary
Evidence	identity/coverage/coordinates/source
Prediction	model output with calibration + OOD
Compare	two samples or two runs
Export	TSV/JSON/report

Execution pipeline

```
FASTA
-> QC
-> AMRFinderPlus annotation
-> normalized AMR features
-> phenotype model
-> calibrated result
-> provenance bundle
-> UI / report
```

The key product difference from a CLI is observability. Every run gets a visible job timeline, CPU/thread count, tool versions, input hash, database version and output artifacts. The user can reproduce or compare runs instead of receiving one opaque answer.

Performance path

Keep Python orchestration and use the C++17 core for expensive sequence indexing or k-mer scanning. Your previous Genome Firewall implementation already demonstrated memory-mapped input, a custom four-way genomic Trie, Bloom filtering, Hamming-distance-one search and lock-free counters; those ideas can inform the new core, but the challenge submission should be a fresh implementation rather than simply shipping the previous repository.

24. Native Apps #4-6 - Sample Vault, Sequence QC, Variant Explorer

The connective tissue

Sample Vault

Sample Vault is the persistent registry. Each sample has identity, raw input URI/hash, source metadata, organism/taxon, accession, QC, analyses and reports. It acts like a file explorer plus scientific metadata browser.

Sequence QC

Sequence QC is a dedicated app for quality distributions across a batch: length, GC, N content, contigs and parser failures. It can also compare the current sample to the training-data QC distribution and flag unusual inputs.

Variant Explorer

Variant Explorer aggregates variant records across samples. Filters can include sample, contig, coordinate, gene, mutation type and annotation source. The app should show a small graph or matrix rather than only a table, allowing the researcher to see whether the same variant recurs across the selected cohort.

App	MVP data sources	Primary output
Sample Vault	sample + sequence + provenance	persistent sample identity
Sequence QC	raw sequence + QC engine	quality report
Variant Explorer	mutation results + annotations	filterable variant matrix

CROSS-APP CONTRACT

All three apps consume the same `SampleRef` object. A selection in one app should be shareable through the OS event bus instead of forcing re-upload.

25. Native Apps #7-8 - Simulation Lab and Radiation Lab

Scientific simulation without pretending to be an experiment

Simulation Lab provides computational “what-if” experiments around abstract biological stress responses. Radiation Lab is a specialized model view. The user can select a reference sequence and a stress class, then run a simplified model that estimates patterns such as relative strand-break burden or repair-stress signals. The result is explicitly labeled as a modeled simulation, not a measured experiment.

The supplied scientific notes distinguish direct and indirect DNA damage and list single-strand breaks, double-strand breaks, base modifications and chromosomal abnormalities as damage classes. They also describe higher-LET radiation as producing denser ionization and more clustered damage than lower-LET radiation.

[NfileciteÖturn1file0ÖL17-L24Ö](#)
[NfileciteÖturn1file0ÖL46-L51Ö](#)

Interface

Control	Output
Stress class	modeled damage profile
Sequence selection	target size/region summary
Simulation seed	reproducible pseudo-random scenario
Run	progress + output artifact
Comparison	side-by-side stress profiles
Evidence	source assumptions + model version

Safety and scientific honesty

The app should not provide actionable exposure recipes or optimize harmful biological outcomes. It can compare conceptual models, visualize repair/damage states, and demonstrate how different assumptions affect a simulated sequence response. Radiation categories and relative-effect claims should be sourced and clearly presented as model assumptions; the supplied notes, for example, summarize high- vs low-LET behavior and comparative effectiveness.

[NfileciteÖturn1file0ÖL46-L51Ö](#)

26. Native Apps #9-11 - Pathogen Atlas, Research Desk, Analysis Studio

Turn outputs into a research environment

Pathogen Atlas

Pathogen Atlas is the browsable taxonomy layer. In the hackathon MVP, keep the benchmark domain bacterial. The UI can reserve categories for viruses and fungi so the architecture is visibly extensible, but do not claim equal model coverage where no training data has been loaded.

Research Desk

Research Desk stores project questions, hypotheses, sample selections, notes, analyses, references and report blocks. Vella can answer within the context of one desk workspace so that the researcher does not need to re-explain the project each time.

Analysis Studio

Analysis Studio is the general scientific notebook surface: upload tabular output, join tables, choose plots, run saved recipes and export figures. For the sprint, a small set of high-value recipes is enough: phenotype distribution, per-antibiotic model performance, QC distribution, mutation frequency and AMR marker frequency.

App	Distinctive interaction
Pathogen Atlas	organism -> samples -> mutations -> AMR findings
Research Desk	question -> evidence -> analysis -> report
Analysis Studio	dataset -> transform -> chart -> export

RESEARCH UX

Clicking a point in a chart should open the exact underlying records. Scientific visualization is only useful when the user can drill from plot -> row -> sample -> evidence.

27. Native Apps #12-14 - Bio Terminal, Report Studio, Forge

Power users and the “OS can build itself” moment

Bio Terminal

The terminal exposes the WSL environment and approved scientific CLI tools. You can support Python, C/C++, Rust, Java, C#, JavaScript/TypeScript and other installed languages, but the important part is that terminal outputs can become artifacts inside Umbrella. A command result can be attached to a sample or report rather than disappearing into scrollbar.

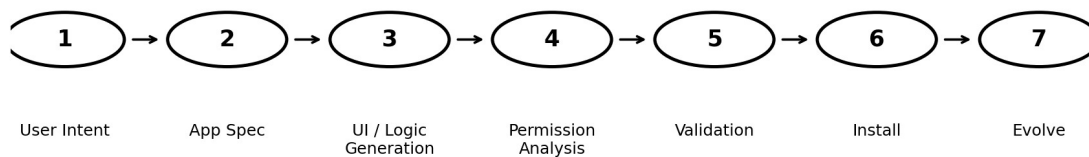
Report Studio

Report Studio composes sections from Sample Vault, QC, mutation, AMR, model and analysis outputs. It should render a clean PDF/HTML report with input hashes, tool versions, database versions, model versions and timestamps.

Forge demo

The hero demonstration should create a useful tool that is obviously not hard-coded. Example request: “Build a bacterial sequence comparator with two sequence selectors, a difference table, a base-level viewer and export.” Forge should show the generated spec, permissions, validation status and then install the app. Follow with: “Add GC-content comparison and save it as a new version.”

UMBRELLA FORGE - SELF BUILDING APPLICATION PIPELINE



Apps are generated against a controlled SDK/runtime, never executed as arbitrary model output.

Use this sequence as the visual story for the generated-app feature.

INTER-APP PAYOFF

Select a sample in Sample Vault -> run Genome Analyzer -> send result to AMR Sentinel -> open a mutation in Variant Explorer -> ask Vella for a summary -> export in Report Studio. That is the full OS loop.

28. Security, Provenance and Kiro Workflow

Make the build auditable and the AI constrained

Security model

- Authentication: normal session/passkey. Voice is optional convenience/signal, not sole authorization.
- App capabilities: sample.read, analysis.run, filesystem.read-scoped, report.write, network.fetch-scoped, etc.
- Generated app sandbox: no arbitrary host execution; shell only through trusted backend job APIs.
- Secrets: server-side environment variables and Kiro-safe secret handling; never embed API keys in browser code.
- Data provenance: input hash, tool version, database version, model version, parameters and output hash.
- Audit trail: user action, app action, job action and generated-app version are timestamped.

Kiro plan

Kiro Specs are suited to greenfield work because they turn requirements into requirements.md, design.md and tasks.md and provide a trackable implementation plan. Steering files provide persistent product/architecture rules; Hooks can run deterministic checks or agent prompts automatically. MCP can expose specialized services to the agent. [8][9][10]

```
.kiro/
  steering/
    product.md
    architecture.md
    scientific-integrity.md
    ui.md
  specs/
    desktop-shell/
    app-runtime/
    forge/
    genome-analyzer/
    mutation-lab/
    amr-sentinel/
    model-pipeline/
  hooks/
    test-on-save.json
    validate-generated-app.json
    provenance-check.json
```

Use Kiro to generate and execute the feature plan, but keep the scientific assumptions in human-reviewed steering/spec files. The goal is to use Kiro as an engineering multiplier, not delegate scientific truth to the coding agent.

29. 18.5-Hour Build Plan and Definition of Done

What to actually ship

SPRINT CONTROL

Freeze scope after hour 14. The final 2.5 hours are for reliability, visual polish, demo reset and hardening only.

Time	Build target
0:00-1:15	Repo, Kiro Spec, architecture skeleton, Docker/WSL sanity check
1:15-3:00	Desktop shell, launcher, window manager, app registry
3:00-4:30	App Runtime contracts + Sample Vault
4:30-6:30	Genome Analyzer + QC job
6:30-8:30	Mutation Lab MVP + result persistence
8:30-11:00	AMR Sentinel + AMRFinderPlus adapter + sample output
11:00-13:00	Logistic model + prediction UI + provenance
13:00-15:00	Vella tools + grounded explanation
15:00-17:00	Forge: generate one native app + validate + install + evolve
17:00-18:30	Report Studio, demo data reset, reliability, polish

Definition of done

- One real bacterial genome can move end-to-end through upload -> QC -> analysis -> AMR -> prediction -> report.
- At least five native scientific apps visibly share sample state.
- Vella can execute at least four typed tools and explain the resulting artifacts.
- Forge can generate one useful app from a natural-language request, validate it, install it and create a versioned modification.
- Every prediction and report displays provenance, model version and dataset version.
- No demo-critical path depends on a live external dataset call; cache the challenge sample locally after acquisition.

FINAL PRODUCT SENTENCE

Umbrella OS is not a genome checker with a desktop attached. It is a programmable scientific operating environment: the OS manages the data, tools, models, provenance and applications - and when a scientific workflow is missing, Forge can build the missing application.

Key references

- [1] BV-BRC Data API - Genome AMR: https://www.bv-brc.org/api/doc/genome_amr
- [2] BV-BRC Data API - Genome and Genome Sequence: <https://www.bv-brc.org/api/doc/genome> and https://www.bv-brc.org/api/doc/genome_sequence
- [3] NCBI AMRFinderPlus: <https://www.ncbi.nlm.nih.gov/pathogens/antimicrobial-resistance/AMRFinder/>
- [4] NCBI AMRFinderPlus GitHub releases / source: <https://github.com/ncbi/amr/releases>
- [5] cAMRah workflow paper: <https://pmc.ncbi.nlm.nih.gov/articles/PMC12910510/>
- [6] HyenaDNA: <https://arxiv.org/abs/2306.15794>
- [7] DNABERT-2: <https://arxiv.org/abs/2306.15006>
- [8] Kiro Specs: <https://kiro.dev/docs/specs/>
- [9] Kiro Hooks: <https://kiro.dev/docs/hooks/>
- [10] Kiro first-project workflow / Steering / Specs / MCP: <https://kiro.dev/docs/getting-started/first-project/>
- [11] XTree: <https://github.com/two-frontiers-project/2FP-XTree>
- [12] Organizer-provided challenge appendix: fixed benchmark/data guidance supplied for this build.